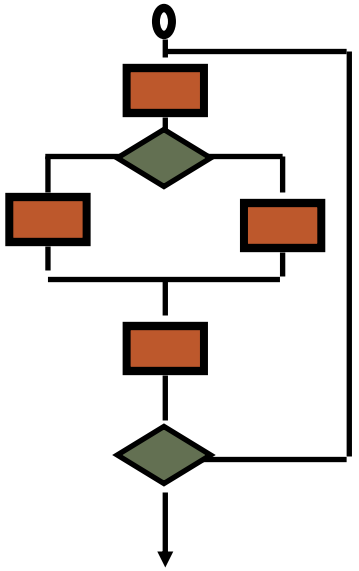
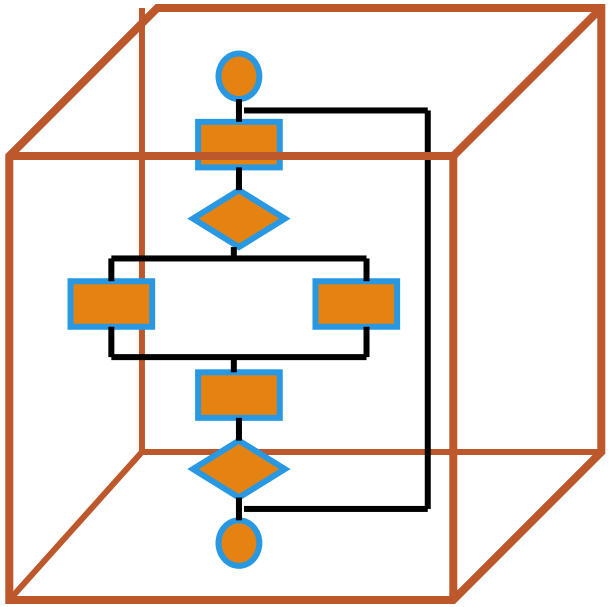
Software Engineering

LECTURE 15: WHITEBOX TESTING

White-Box Testing





... our goal is to ensure that all statements and conditions have been executed at least once ...

White-Box

- White box testing, sometimes called glass-box and structural testing.
- Various aspects like Statement Coverage Criteria, Edge coverage, Condition Coverage, Path Coverage are defined mathematically and test set is designed accordingly.
- There are no algorithms for generating white box test data. However the checklist might help:
  - Has every line of code been executed at least once by test data.
  - Have all default paths been traversed at least once.
  - Have all significant combinations of multiple conditions been identified and
    - Have all logical decisions exercised on their true and false sides.
  - Have all loops executed at their boundaries and within their operational bounds.
  - Have internal data structures validated▪

Control flow graph (CFG)

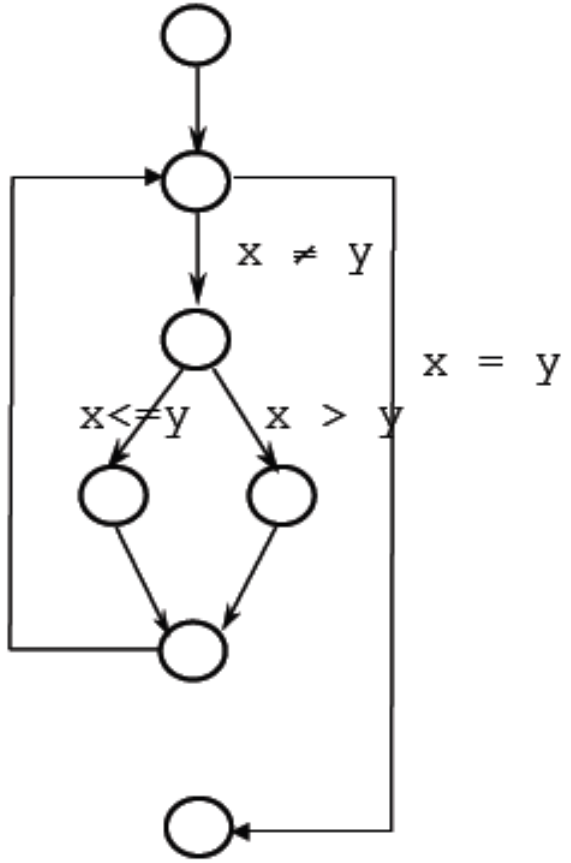
Gives an abstract representation of the code

Directed Graph

Nodes are blocks of sequential statements

Edges are transfers of control

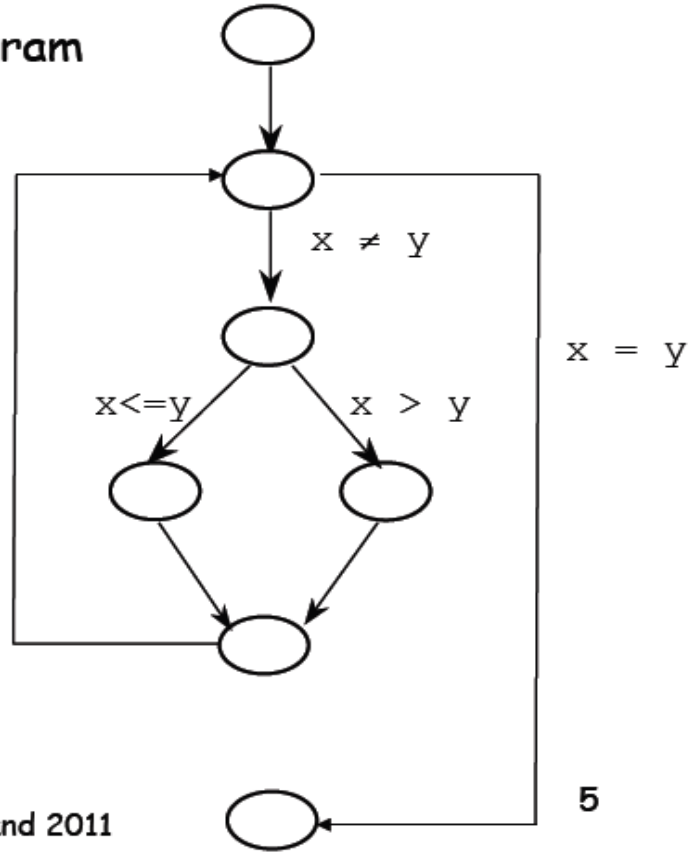
- May be labeled with predicate representing the condition of control transfer



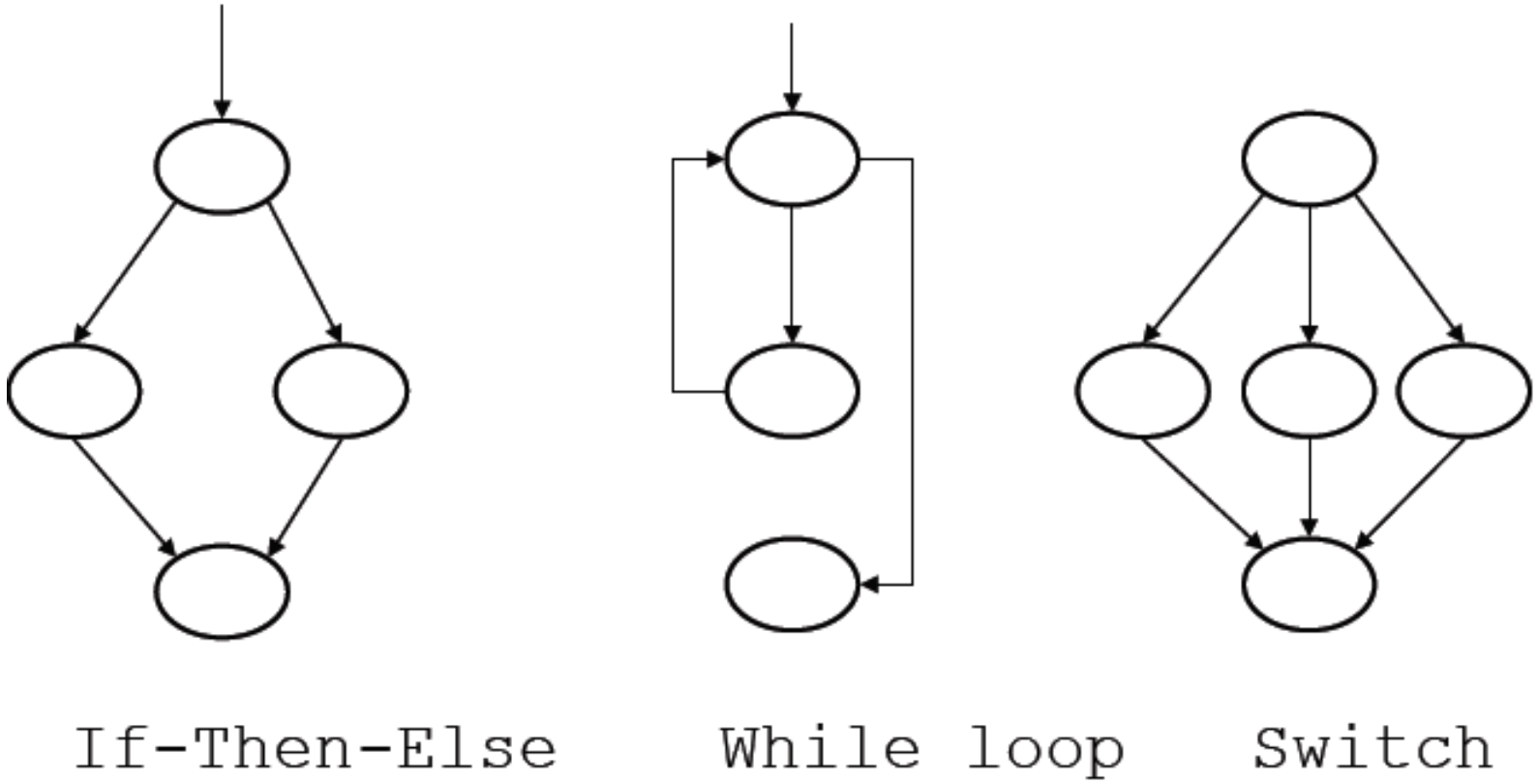
Control Flow Graph - Example

Greatest common divisor (GCD) program

```
read(x);
read(y);
while x ≠ y loop
   if x>y then
    x := x - y;
   else
    y := y - x;
   end if;
end loop;
gcd := x;
```



Basics of CFG: Patterns



If-Then-Else While loop Switch

Example-CFG

A = 10

IF B > C

THEN A = B

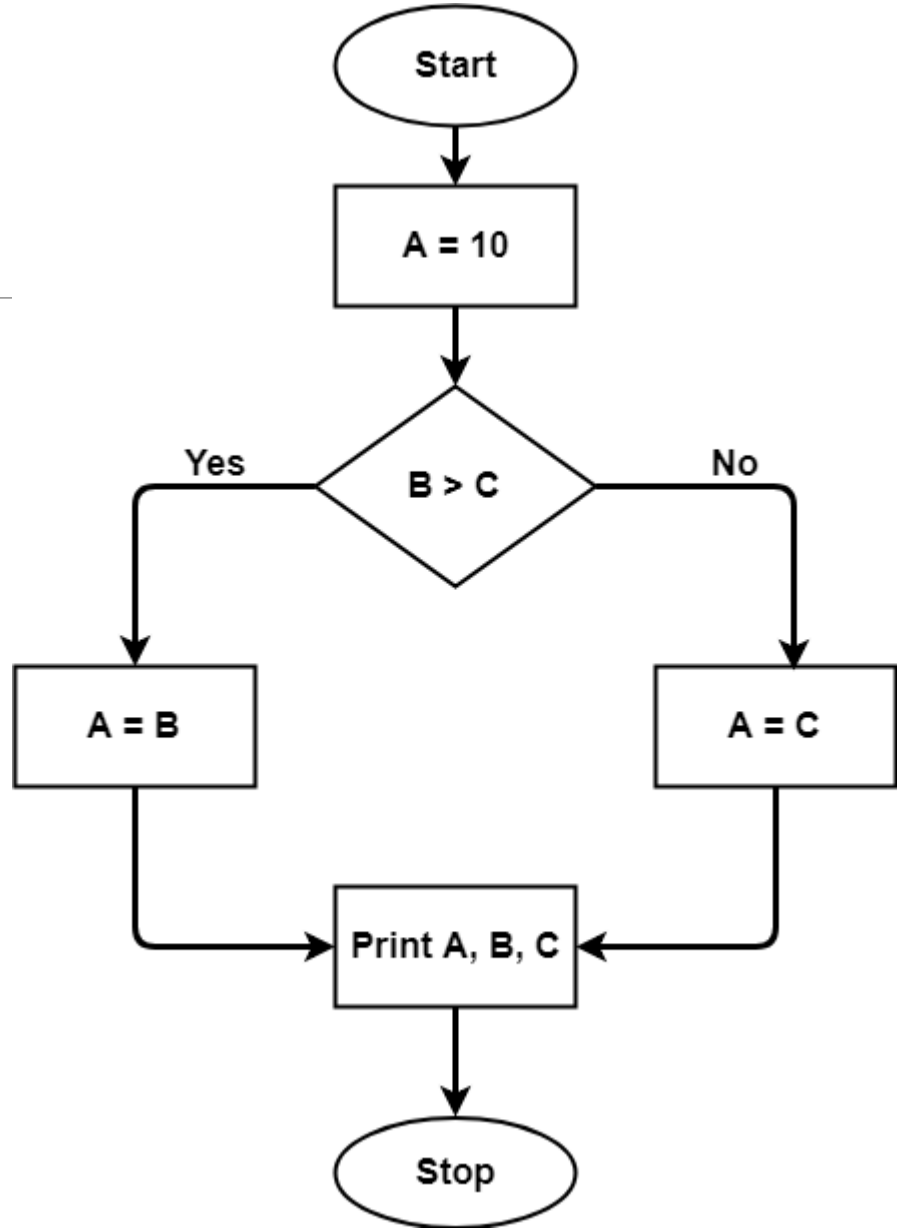
ELSE A = C

ENDIF

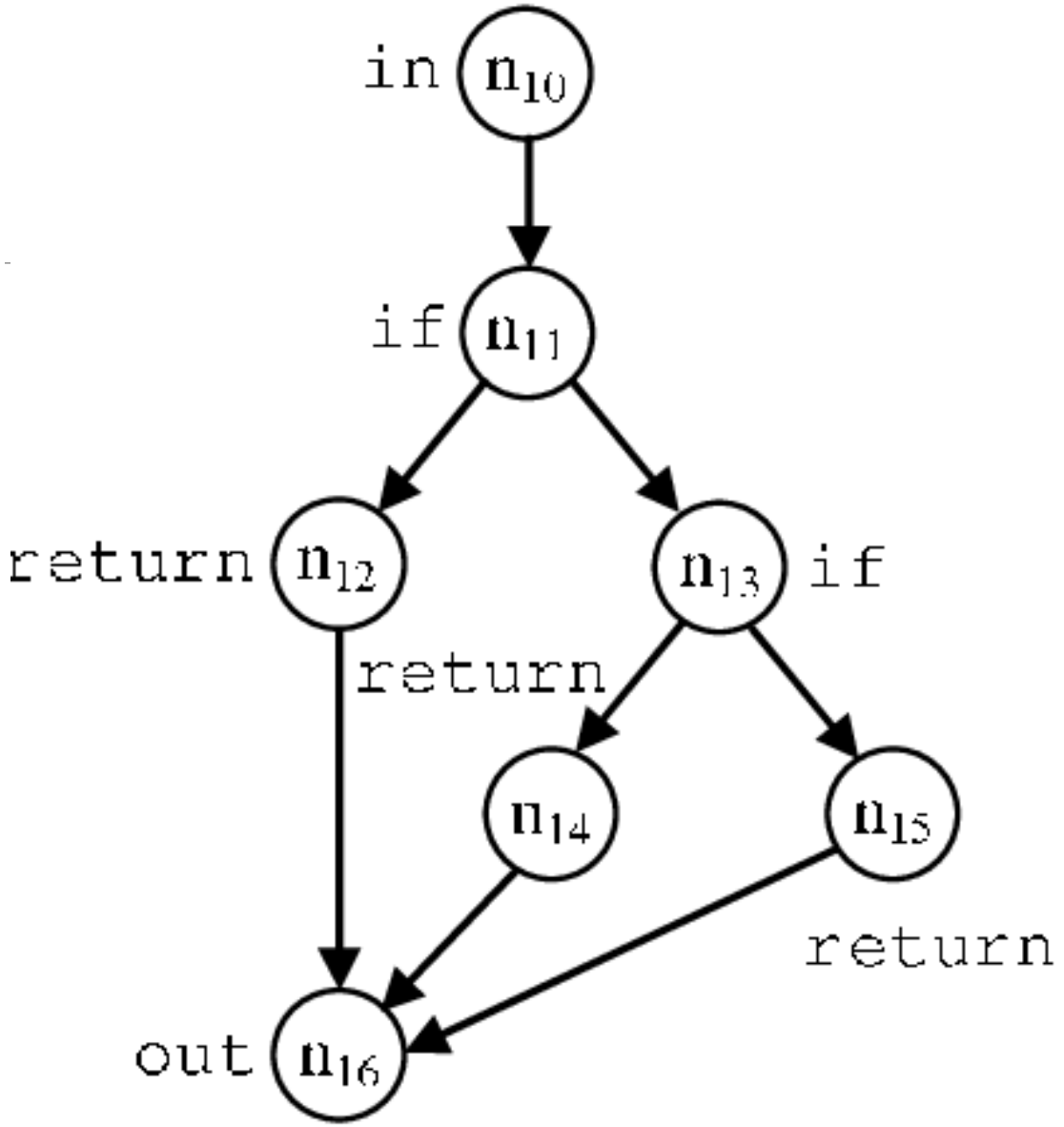
Print A

Print B

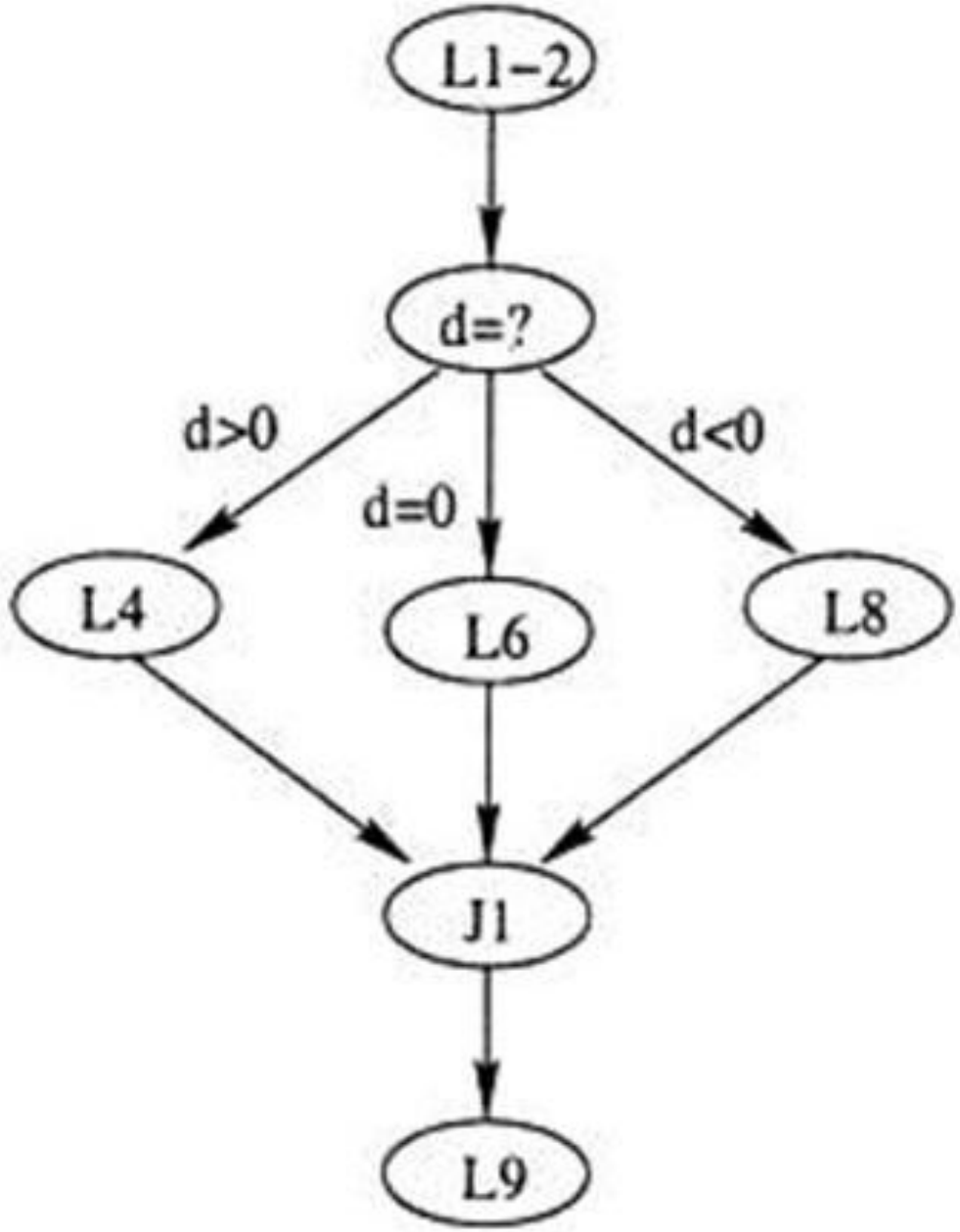
Print C



```
int isabsequal(int x, int y)
{
   if (x == y)
      return 1;
   else if (x == -y)
      return -1;
   else
      return 0;
}
```



```
L1:     input(a, b, c);
L2:     d ← b * b - 4 * a * c;
L3:     if ( d > 0 ) then
L4:         r ← 2
L5:     else_if ( d = 0 ) then
L6:         r ← 1
L7:     else_if ( d < 0 ) then
L8:         r ← 0;
L9:     output(r);
```



Control Flow Coverage

Depending on the (adequacy) criteria to cover (traverse) CFGs, we can have different types of control flow coverage:

a) **Statement/Node Coverage**
b) **Edge Coverage**
c) **Condition Coverage**
d) **Path Coverage**

Discussed in detail next…

(1) Statement/Node Coverage

1) Hypothesis: Faults cannot be discovered if the statements containing them are not executed
2) Statement coverage criteria: Equivalent to covering all nodes in CFG
3) Executing a statement is a weak guarantee of correctness, but easy to achieve
4) Several inputs may execute the same statements
5) An important question in practice is:
    1) how can we minimize (the number of) test cases so we can achieve a given statement coverage ratio?

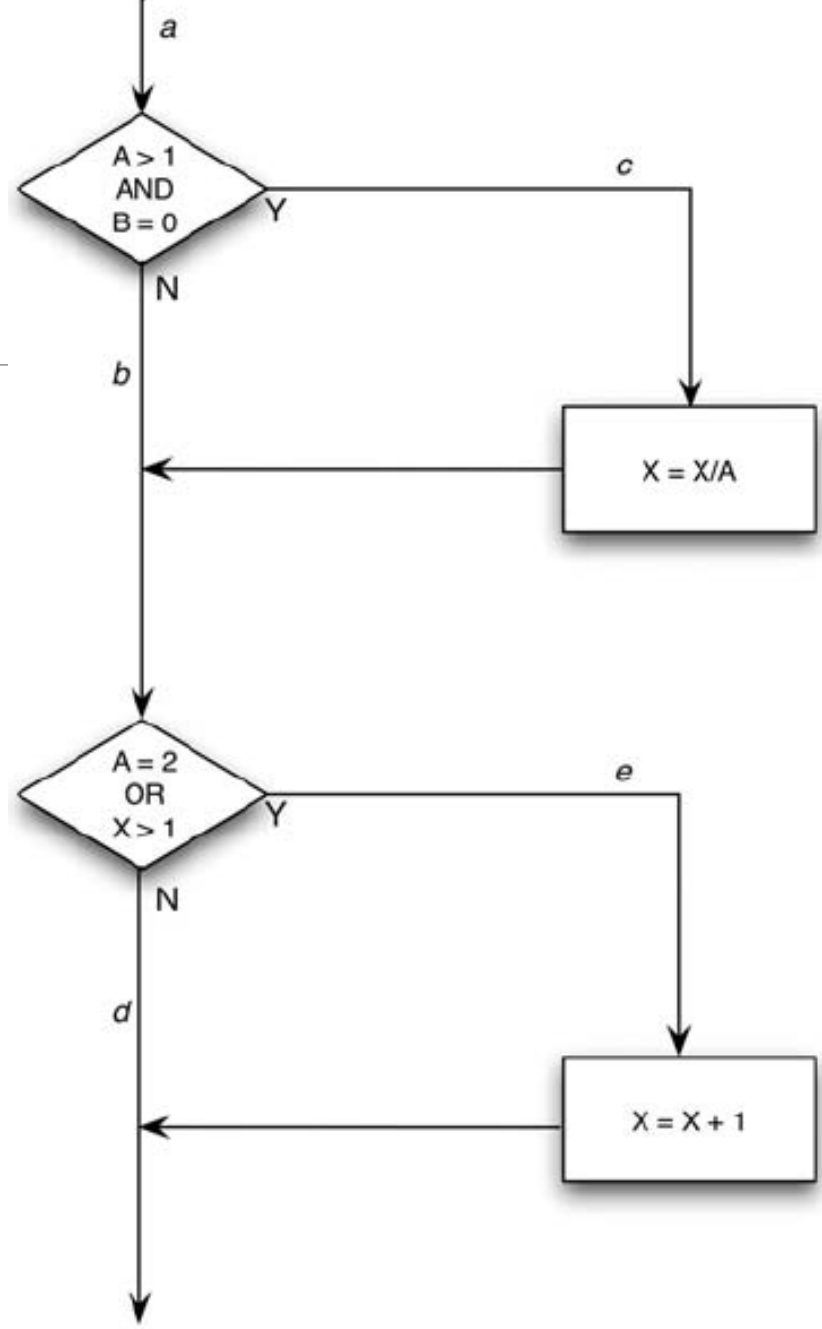
Statement coverage

```
public void foo(int a, int b, int x) {
    if (a>1 && b==0)  {
        x=x/a;
    }
    if (a==2 || x>1)  {
        x=x+1;
    }

}
```

Execute every statement in the program at least once

For path: {a,c,e}

- (A, B, X) := (2, 0, 3) or
- (A, B, X) := (2, 0, any X)



Incompleteness of Statement/Node Coverage

Though often used, statement coverage is a weak guarantee of fault detection

```
if x < 0 then
      x := -x;
end if
z := x;
```

A negative x would result in the coverage of all statements.

But not exercising x >= 0 is an important case.

Doing nothing for the case x >= 0 may turn out to be wrong and need to be tested.

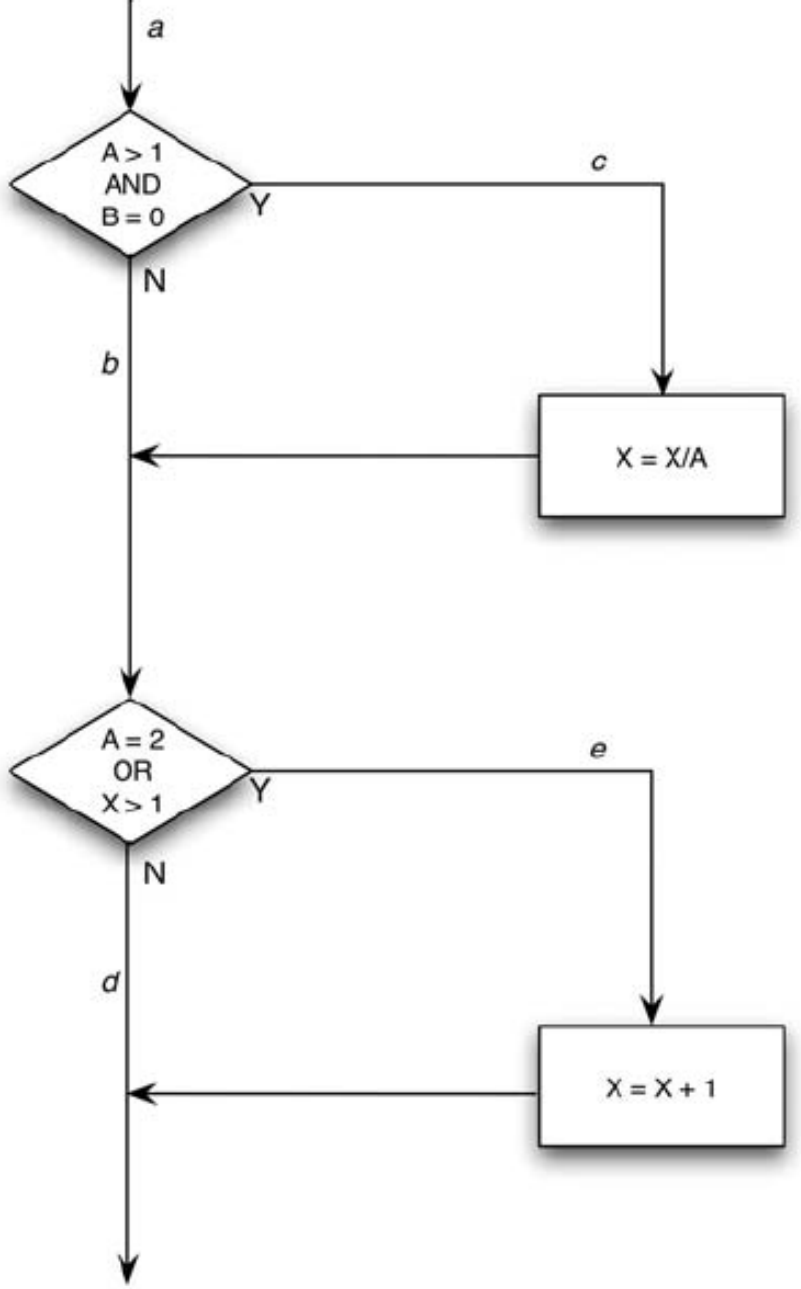
Imagine z := z/x; // divide by 0

Statement coverage

```
public void foo(int a, int b, int x) {
    if (a>1 && b==0)  {
        x=x/a;
    }
    if (a==2 || x>1)  {
        x=x+1;
    }
}
```

Problems:

- Line 2: if the requirement was || rather than &&
- Line 5: no difference if x > 1 condition was correct or even missing
- Didn't test the path {a, b, d} in which x is unchanged
- Conclusion: Same test (a=2, b=0) tests all statements but not all conditions.



$$\textbf{Statement Coverage} = \frac{\textit{Number of executed statements}}{\textit{Total number of statements}} \times 100$$

Prints (int a, int b)

{ int result = a+ b;

If (result> 0)

Print ("Positive",result)

Else Print ("Negative", result) }

```
1  Prints (int a, int b) {
2  int result = a+ b;
3  If (result> 0)
4      Print ("Positive", result)
5  Else
6      Print ("Negative", result)
7  }
```

If A = 3, B = 9

The statements marked in yellow color are those which are executed as per the scenario

Number of executed statements = 5, Total number of statements = 7

Statement Coverage: 5/7 = 71%

Scenario 2:

If A = -3, B = -9

The statements marked in yellow color are those which are executed as per the scenario.

Number of executed statements = 6

Total number of statements = 7

Statement Coverage: 6/7 = 85%

```
1  Prints (int a, int b) {
2  int result = a+ b;
3  If (result> 0)
4      Print ("Positive", result)
5  Else
6      Print ("Negative", result)
7  }
```

2. Decision (or branch)/Edge Coverage

Decision coverage reports the true or false outcomes of each Boolean expression.

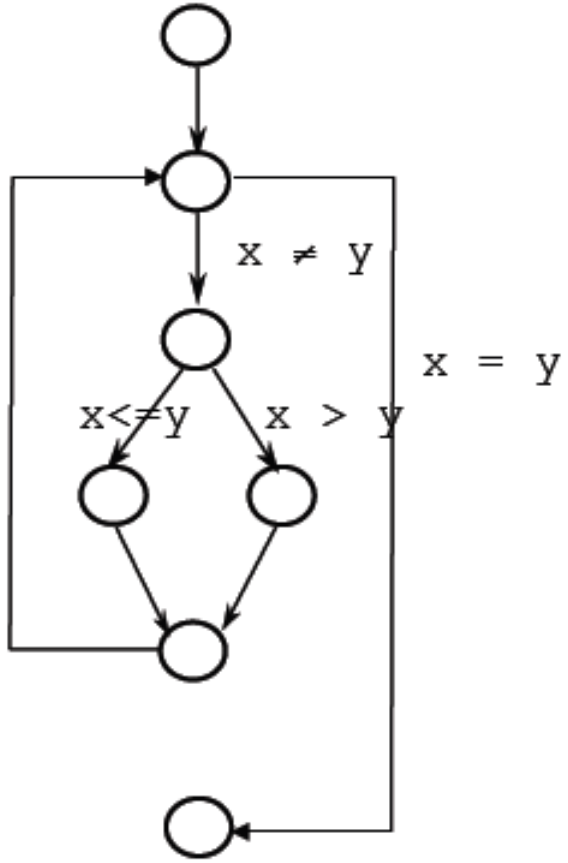
In this coverage, expressions can sometimes get complicated. Therefore, it is very hard to achieve 100% coverage

2. Decision (or branch) Coverage

Select a test set T such that, by executing P for each test case t in T, each edge of P's control flow graph is traversed at least once

We need to exercise all conditions that traverse the control flow of the program with true and false values

Also known as branch or edge coverage



Decision Coverage

If there is an error for x >= 0, it will be discovered by decision coverage
*

```
if x < 0 then
        x := -x;
end if
z := x;
```

If (A or B)

TF and FF will toggle the decision *

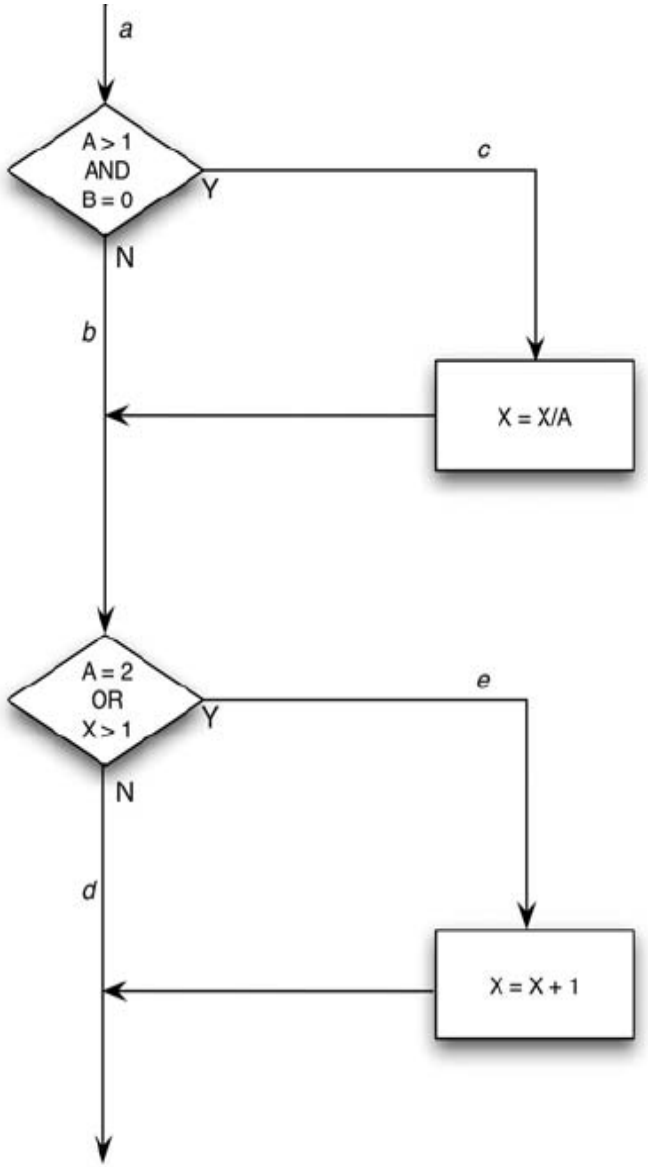
Decision coverage

You must write enough test cases that each decision has a T and a F outcome at least once.

- ace & abd or acd & abe
  - (2, 0, 4) & (1, 1, 1) {TT & FF}
  - (3, 0, 3) & (2, 1, 1) {TF & FT}

Problem

- If the condition x > 1 had a fault, e.g., x < 1, the mistake would not be detected



$$\text{Decision coverage} = \frac{\text{Number of decision outcomes exercised}}{\text{Total number of decision outcomes}} \times 100\%$$

Demo(int a)

{ If (a> 5)

a=a*3

Print (a)

}

Value of a is 2

Value of a is 6

In both cases 50% coverage

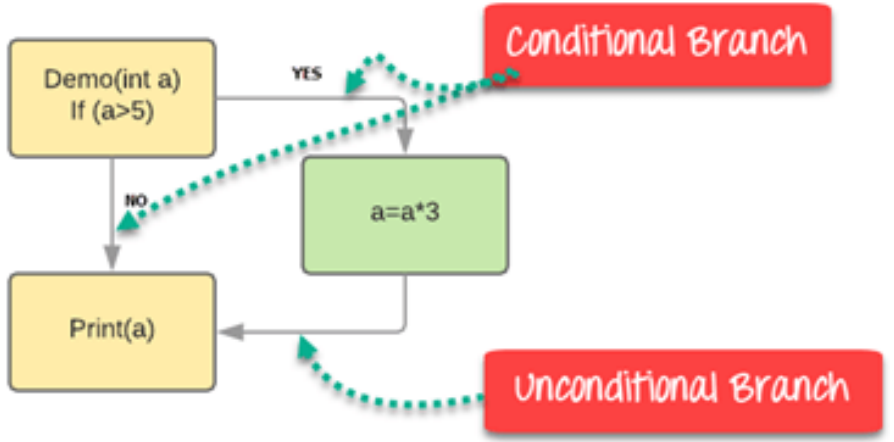
```
1  Demo(int a) {
2          If (a> 5)
3              a=a*3
4          Print (a)
5   }
```

```
1  Demo(int a) {
2          If (a> 5)
3              a=a*3
4          Print (a)
5   }
```

$$Branch\ Coverage = \frac{Number\ of\ Executed\ Branches}{Total\ Number\ of\ Branches}$$

```
Demo(int a)
{
If (a> 5)
a=a*3
Print (a)
 }
```

Branch Coverage will consider unconditional branch as well



Test Case	Value of A	Output	Decision Coverage	Branch Coverage
1	2	2	50%	**33%**
2	6	18	50%	**67%**

Branch coverage advantages

Allows you to validate-all the branches in the code

Helps you to ensure that no branch leads to any abnormality of the program's operation

Branch coverage method removes issues which happen because of statement coverage testing

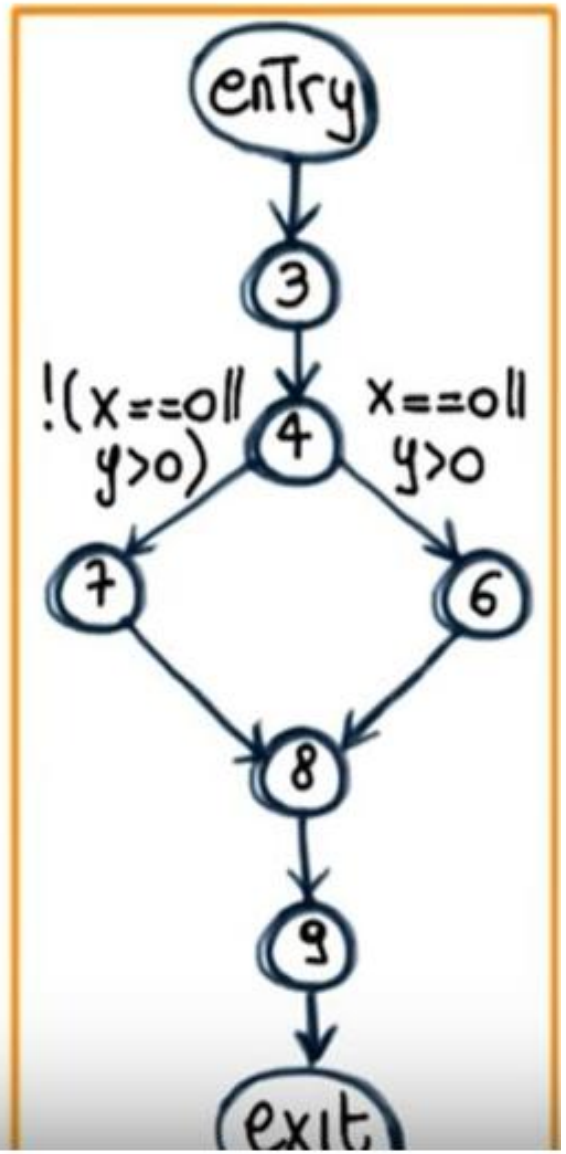
Allows you to find those areas which are not tested by other testing methods

It allows you to find a quantitative measure of code coverage

Branch coverage ignores branches inside the Boolean expressions

Imagine following Example

```
1. void main(){
2.    float x,y;
3.    read(x);
4.    read(y);
5.    if((x==0)||(y>0))
6.       y=y/x;
7.    else x=y+2;
8.    write(x);
9.    write(y);
10. }
```



Imagine
x = 1, y = 0
x = 2, y = 2

What is the decision coverage of the above scenario

3. Condition Coverage

Conditional coverage or expression coverage will reveal how the variables or subexpressions in the conditional statement are evaluated. For example, if an expression has Boolean operations like AND, OR, XOR, which indicated total possibilities.

Condition Coverage (CC) Criterion:

- Select a test set T such that, by executing P for each element in T, each edge of P's control flow graph is traversed, and all possible values of the **constituents** of **compound conditions** are exercised at least once

If (A or B)

TF/FT and FF will toggle the conditions

Condition coverage

You must write enough test cases that each condition has a T and a F outcome at least once i.e.

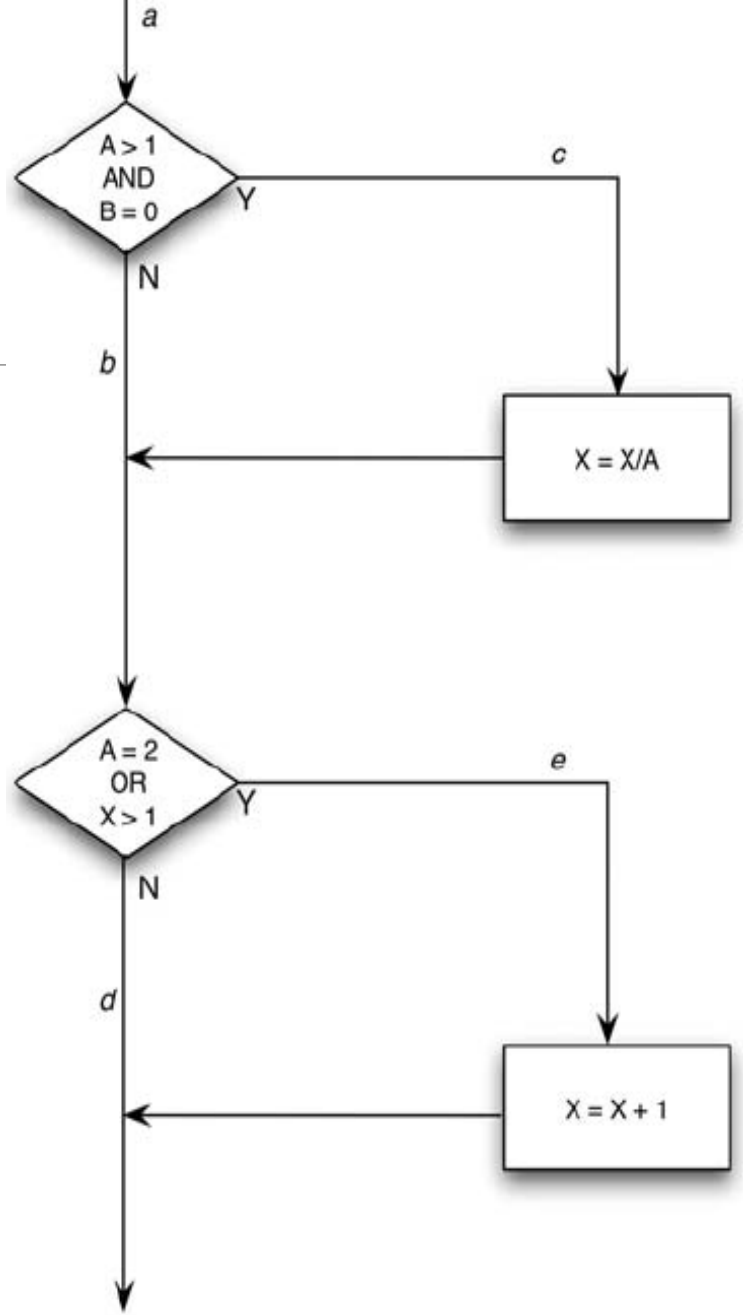
A > 1, A <= 1, B = 0, B != 0

A = 2, A != 2, X > 1, X <= 1

The following 2 TCs will do that:

1. A=2, B=0, X=4 ace
2. A=1, B=1, X=1 abd

.



Problem with Condition Coverage

May not cover All decisions

Example

A. **If (A && B)**

1) A: True, B: False
2) A: False, B: True

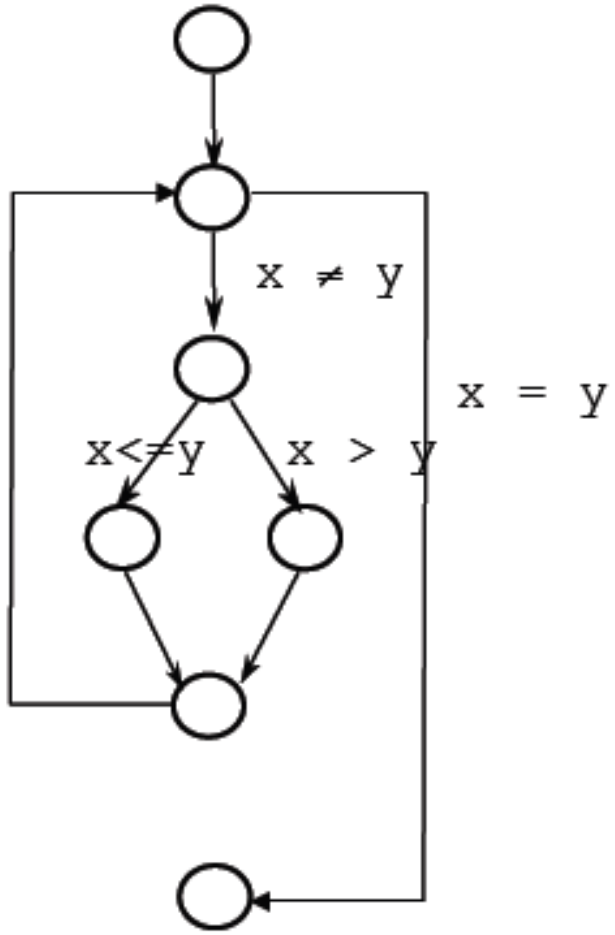
B. **If (A || B)**

1) A: True, B: False
2) A: False, B: True

*Any problem with above 4 TCs?

In case A **True** value of Decision is **not achieved**

In case B **False** value of Decision is **not achieved**



4. Path Coverage

Path Coverage Criterion: Select a test set T such that, by executing P for each test case t in T, all paths leading from the initial to the final node of P's control flow graph are traversed

In practice the number of paths is too large, if not infinite

- (e.g., when we have loops)

Some paths are infeasible

- e.g., not practical given the system's business logic

Path Coverage – Dealing with Loops

In practice, however, the number of paths can be too large, if not infinite (e.g., when we have loops) → Impractical

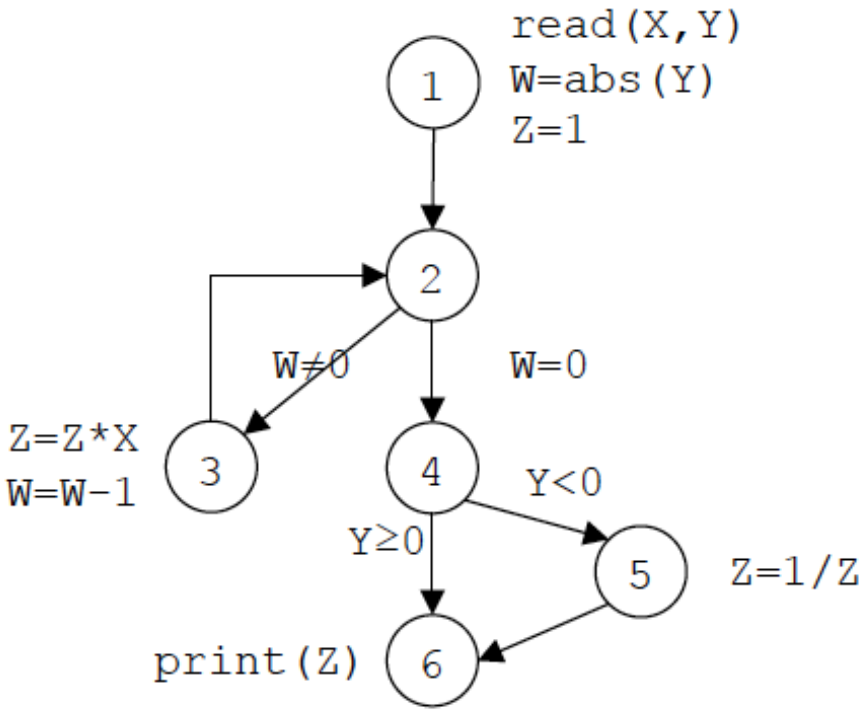
A pragmatic heuristic: Look for conditions that execute loops

- Zero times
- A maximum number of times
- A average number of times (statistical criterion)

For example, in the array search algorithm

- Skipping the loop (the table is empty)
- Executing the loop once or twice and then finding the element*
- Searching the entire table without finding the desired element

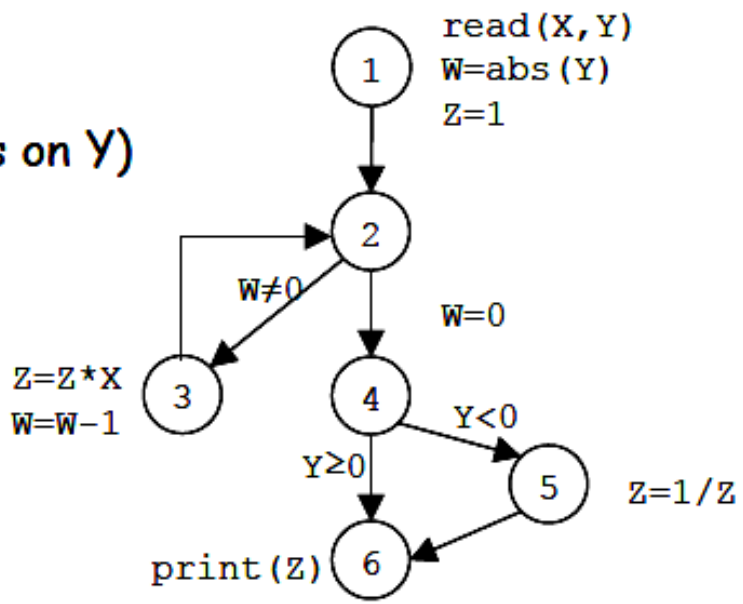
```
Program computing
Z=X^Y

BEGIN
  read (X, Y) ;
  W = abs(Y) ;
  Z = 1 ;
  WHILE (W <> 0) DO
    Z = Z * X ;
    W = W - 1 ;
  END
  IF (Y < 0) THEN
    Z = 1 / Z ;
  END
  print (Z) ;
END
```



Path Coverage: Loops Another Example

Path Coverage - Example

- All paths
  - Infeasible path
    - 1 → 2 → 4 → 5 → 6, Why infeasible?
    - The way Y and W relate.
  - Potentially large number of paths (depends on Y)
    - As many ways to iterate
      2 → (3 → 2)* as values of Abs(Y)
- All branches
  - Two test cases are enough
    - Y<0 : 1 → 2 → (3 → 2)+ → 4 → 5 → 6
    - Y>0 : 1 → 2 → (3 → 2)* → 4 → 6
- All statements
  - One test case is enough
    - Y<0 : 1 → 2 → (3 → 2)+ → 4 → 5 → 6



Control Flow Coverage: Reachability

Not all statements are usually reachable in real-world programs

It is not always possible to decide automatically if a statement is reachable and the percentage of reachable statements

When one does not reach a 100% coverage, it is therefore difficult to determine the reason

Tools are needed to support this activity but it cannot be fully automated

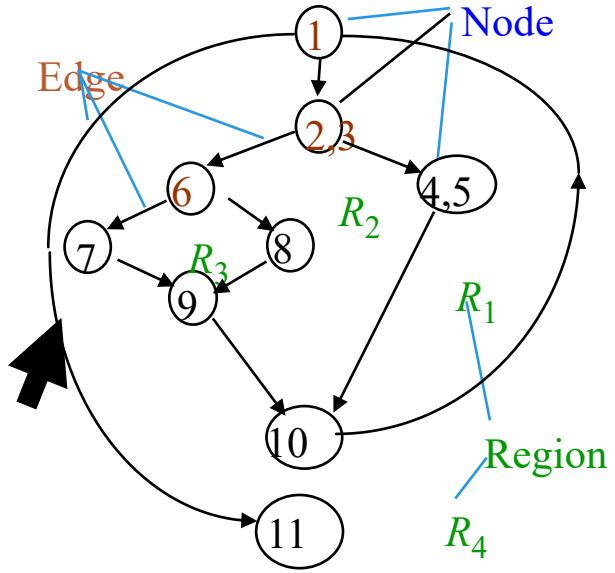
Research focuses on search algorithms to help automate coverage

Control flow testing is, in general, more applicable for testing small pieces of code

Cyclomatic Complexity

- Cyclomatic complexity provides a quantitative measure of the logical complexity of a program
  - It is the number of tests that must be conducted to assure that all statements have been executed at least once
- Cyclomatic complexity, V(G) is given by:

(1) V(G) = E - N + 2
*where **E** is the number of **edges** and **N** number of **nodes** (11-9 +2 = **4**)*
*(2) V(G) = P + 1, where **P** is the number of **predicate nodes** (3+1 = **4**)*
(3) V(G) = number of regions (**4**)



Example-CC

A = 10

IF B > C

THEN A = B

ELSE A = C

ENDIF

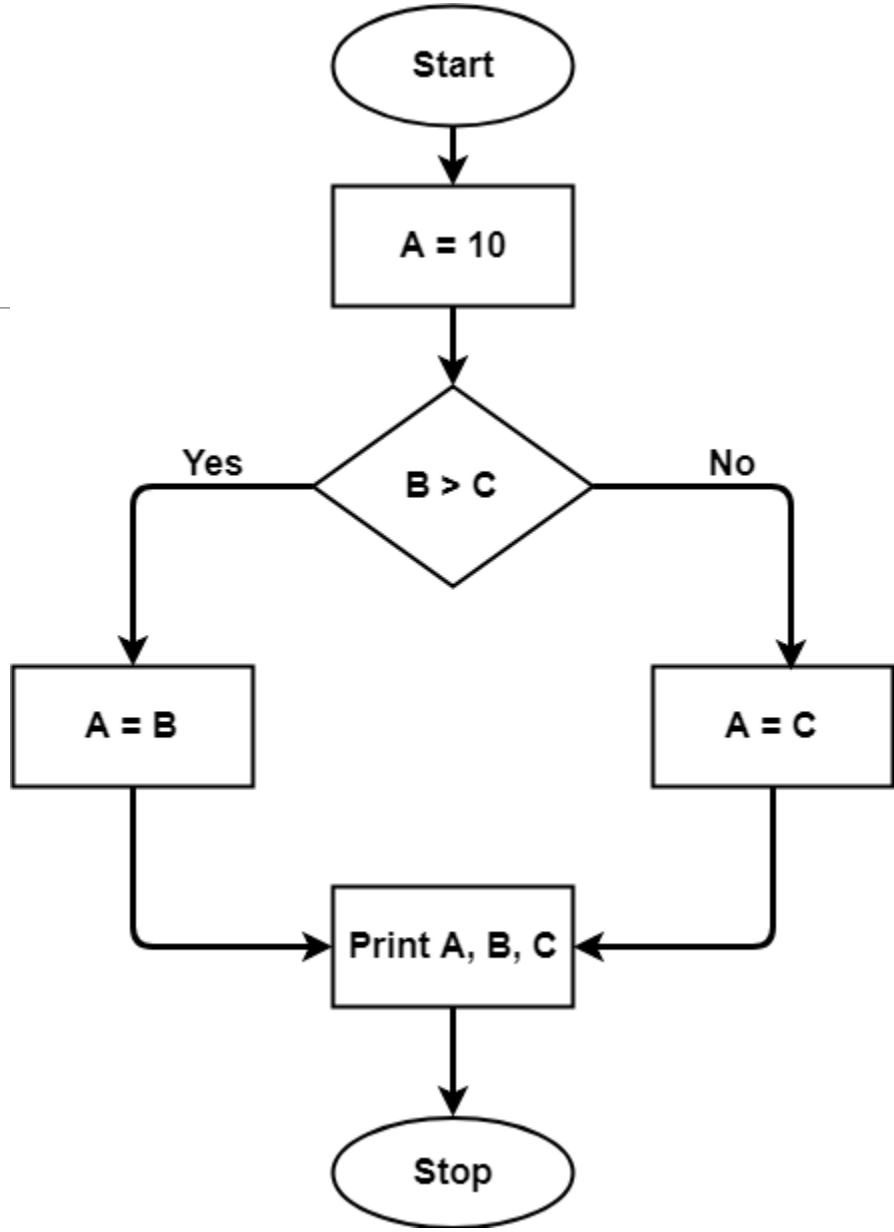
Print A

Print B

Print C

The graph shows seven shapes (nodes), seven lines(edges), hence

cyclomatic complexity = 7-7+2 = 2

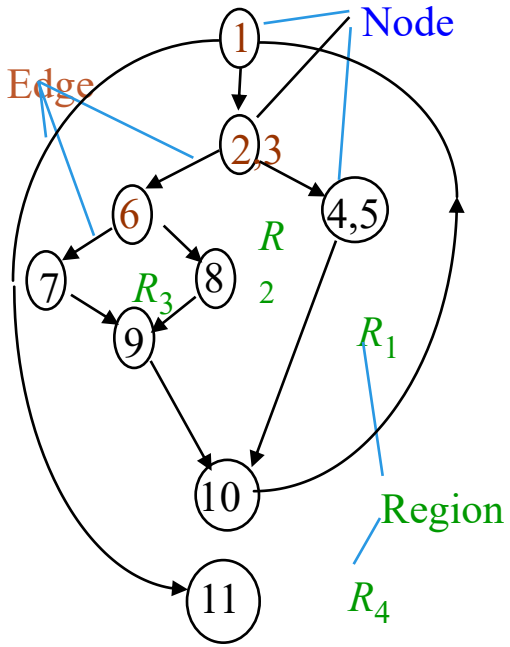


Independent Paths

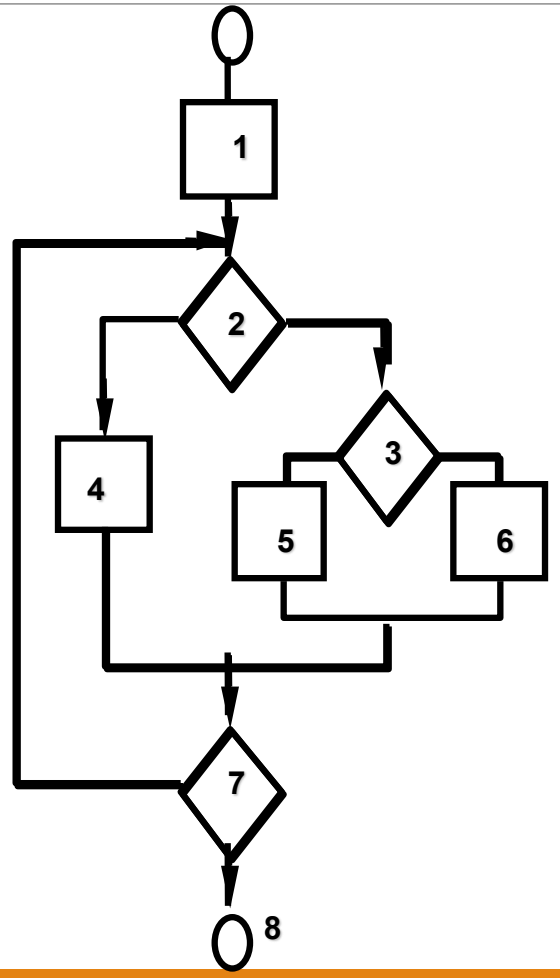
- The value for cyclomatic complexity defines the number of independent paths in the basis set of a program
- An independent path is any path through the program that introduces at least one new set of processing statements or a new condition
- An independent path must move along at least one edge that has not been traversed before the path is defined
- A **set of independent paths** for a flow graph composes a **basis set.**

Deriving Test Cases

- Using the design or code, draw a corresponding flow graph
  you don't need a flow chart,
- Determine the cyclomatic complexity (even without developing flow graph: count each simple logical test (compound tests as 2 or more) + 1
- Determine a basis set of paths
- Prepare test cases that will force execution of paths in the basis set
- basis path testing should be applied to critical modules



Basis Path Testing



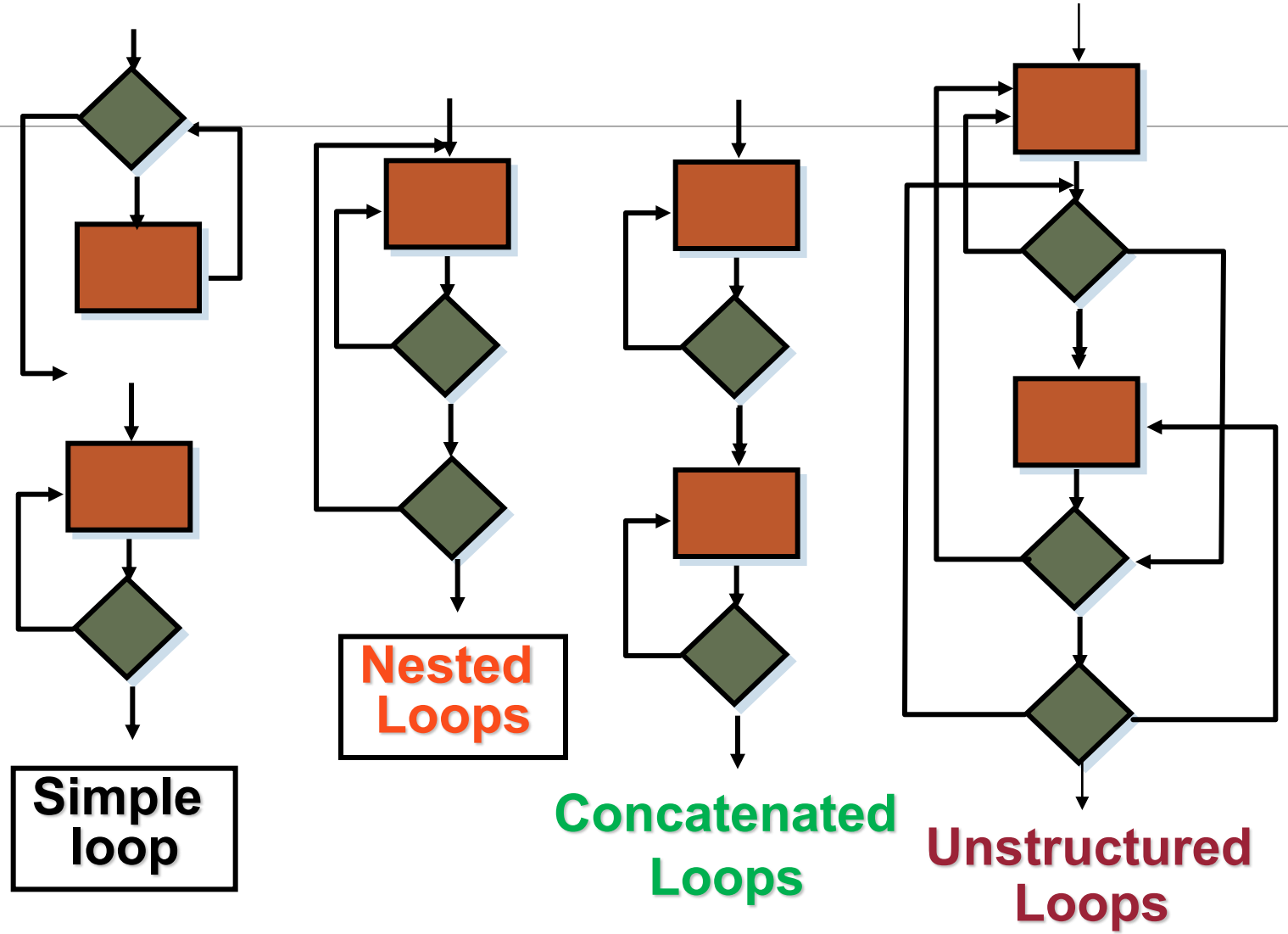
derive the independent paths:

Since V(G) = 4, there are four paths

Path 1: 1,2,3,6,7,8
Path 2: 1,2,3,5,7,8
Path 3: 1,2,4,7,8
Path 4: 1,2,4,7,2,4,...7,8

derive test cases to exercise these paths.

Control Structure Testing: Loop



Loop Testing: Simple Loops

Minimum conditions—Simple Loops

1. **skip the loop entirely**
2. **only one pass through the loop**
3. **two passes through the loop**
4. **m passes through the loop m < n***
5. **(n-1), n, and (n+1) passes through the loop**

*where n is the maximum number of allowable passes

Loop Testing: Nested Loops

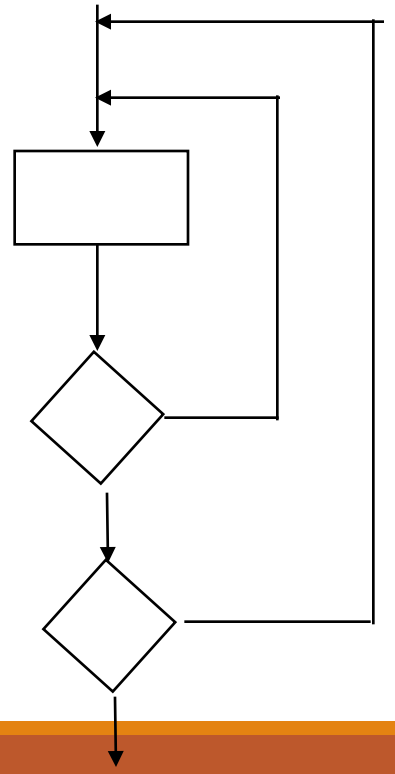
Nested Loops

(1) Start at the innermost loop. Set all outer loops to their minimum iteration parameter values.

(2) Test the min+1, typical, max-1 and max for the innermost loop (while holding the outer loops at their minimum values).

(3) Move out one loop and set it up as in step 2, holding all other loops at typical values.

Continue this step until the outermost loop has been tested.



Concatenated Loops

If the loops are independent of one another
then treat each as a simple loop
else
treat as nested* loops
endif

** for example, the final loop counter value of loop 1 is used to initialize loop 2*

For Other control structures DIY

Testing Phase

What content should be included in a software test plan?

- Testing activities and schedule
- Testing tasks and assignments
- Selected test strategy and test models
- Test methods and criteria
- Required test tools and environment
- Problem tracking and reporting
- Test cost estimation

Test Execution

- using manual approach
- using an automatic approach
- using a semi-automatic approach

Basic activities in test execution:

- Select a test case
- Set up the pre-conditions for a test case
- Set up test data
- Run a test case following its procedure
- Track the test operations and results
- Monitor the post-conditions of a test case & expected results
- Verify the test results and report the problems if there is any
- Record each test execution

Bug Report

Problem ID: ***B-101*** ***current software name:*** ***Windows Calculator***
Release and Version No.: ***V 1.0***
Test type: ***Manual*** ***Reported by:*** ***Junaid***
Reported date: ***26th April 2005*** ***Test case ID:*** ***T-101***
Subsystem (or module name): ***Calculation*** ***Feature Name (or Subject):*** ***Add Numbers***
Problem type (REQ, Design, Coding,): ***Coding***
Problem severity (Fatal, Major, Minor,): ***Major***
Problem summary and detailed description:
On adding two numbers the result is not correct.
Cause analysis: ***NA***
How to reproduce? Why Required:
Attachments: Nil


Thank
You